

Progetto d'esame — Pipeline realizzativa e piano di studio

B031278 *Deep Learning*, Fall 2025 — MSc in Artificial Intelligence, Università di Firenze

Riproduzione di **Kavaki & Mandel (2025)**, *Audio Sparse-Transformer for Speech Classification*, con **Gazneli et al. (2022)** come baseline

Docente: Paolo Frasconi (DINFO) · Documento di lavoro personale · v2.1 — 18 agosto 2026

v2.0: metodo di Kavaki & Mandel ricavato dal PDF integrale; ancestor corretto (*SparseFormer*, non la linea importance-map);

Fase 3 semplificata a early conv + dense transformer.

v2.1: §3.4 aggiornata — quasi tutte le ambiguità risolte leggendo il codice ufficiale di *SparseFormer* e di *EAT*.

Questo documento è il piano — cosa fare e in che ordine. Per i dettagli di metodo, le formule e le evidenze il riferimento è `Manuale_tecnico.pdf`; per la cronaca delle decisioni e degli errori, `Diario_di_bordo.pdf`.

1 Vincoli d'esame: cosa determina ogni scelta

Tutte le decisioni tecniche di questo documento discendono da cinque vincoli fissati dalla pagina del corso. Vanno tenuti presenti perché diversi di essi sono controintuitivi.

Vincolo	Contenuto e conseguenza operativa
Concordare a ricevimento	«You should discuss it with me during office hours and I will give you the details of what should be done.» Il testo dell'assegnazione è la proposta; i <i>dettagli operativi</i> (quale task esatto di Speech Commands, quali tabelle riprodurre, quanto ridurre) li fissa il docente. Ricevimento: lunedì 10:45–12:45, S. Marta.
Niente relazione	Si consegna <i>solo il codice</i> più brevi istruzioni di riproduzione. Il <code>README.md</code> è quindi l'unico documento scritto valutato: va trattato come tale.
Modalità di consegna	Repository privato su Codeberg con l'utente <code>dl.unifi</code> invitato come membro. In alternativa zip via email, o link <code>https://0x0.st/</code> se supera 1 MB. Mai i dati : per quelli basta un link.
Presentazione 30'	Deve contenere: motivazione del problema <i>nel contesto della letteratura</i> , <i>derivazione tecnica</i> dei metodi, descrizione <i>dettagliata</i> del lavoro sperimentale. Più domande individuali su tutto il programma.
Riduzione autorizzata	«Va comunque bene ridurre la dimensione dei datasets e/o dei modelli, se le risorse di calcolo non fossero sufficienti.» Riduci <i>dataset e modello</i> , mai il protocollo di valutazione .

L'AVVERTIMENTO CHE VALE PER QUESTO PROGETTO PIÙ CHE PER ALTRI

«In some cases, studying the references covered in class may be sufficient but in other cases, **especially when dealing with the details of the experimental procedures, reading other ancestors in the citation graph may be necessary.**» Nel tuo caso non è un consiglio generico: il paper da riprodurre è un 4 pagine ICASSP *closed access e senza codice*, e il metodo vero è quasi certamente nei lavori precedenti degli stessi autori (§3).

Da attivare subito, indipendentemente da tutto il resto: il **form Google per le risorse di calcolo** indicato nella sezione *Computational resources* della pagina del corso.

2 I due paper e come si incastrano

2.1 Gazneli et al. 2022 — EAT (arXiv:2204.11479)

Accesso libero, **codice ufficiale disponibile** (github.com/Alibaba-MIIL/AudioClassification). Rete di classificazione audio **end-to-end sulla forma d'onda grezza**, senza spettrogramma: stack di convoluzioni 1D (depth-wise con kernel lunghi sull'asse temporale, poi 1×1 channel-wise, filtro passa-basso fisso, downsampling di un fattore ≈ 256) seguito da un encoder transformer e da una testa lineare.

Variante	Canali	Layer / teste	Embedding	Parametri
EAT-S	16	4 / 8	128	5,3 M
EAT-M	32	6 / 16	256	25,5 M

Il contributo dichiarato non è l'architettura ma le **augmentation audio**. Due famiglie:

- **Label-mixing** — *Mixup*, *FreqMix*, *PhaseMix*, *TimeMix*/*CutMix* in versione 1D. **Attenzione: i nomi ingannano.** Il «mixup» di EAT è in realtà il *between-class learning* di Tokozume et al., con bilanciamento in dB e rinormalizzazione dell'energia; e *PhaseMix* **non mescola l'ampiezza affatto**. Formule in [Manuale_tecnico.pdf §6.3](#).
- **Label-preserving** — gain, rumore bianco e colorato, time shift, filtraggio, inversione di polarità, time masking, quantizzazione, resampling.

Ricetta di training: **AdamW**, lr $5 \cdot 10^{-4}$, schedule **one-cycle**, weight decay 10^{-5} , **EMA** 0,995, label smoothing 0,1, **BCE** quando si usa il mixing.

Dataset	Modello	Risultato riportato
ESC-50 (from scratch)	EAT-S	92,15 %
ESC-50 (pretrain AudioSet)	EAT-M	96,3 %
UrbanSound8K (from scratch)	EAT-S	85,5 %
SpeechCommands	EAT-S	98,15 %
AudioSet	EAT-M	42,6 mAP

2.2 Kavaki & Mandel 2025 — Audio Sparse-Transformer (ICASSP)

Cinque pagine, closed access, nessun codice pubblico. Idea: sostituire la lunga sequenza di frame temporali con una **sequenza corta di token latenti**. Il metodo completo è in §3.

Risultati riportati su Google Speech Commands V2: **96,88 %** con 2,87 M parametri e 0,055 G FLOPs, cioè **−85,25 % di FLOPs rispetto a EAT-S** e **−91,47 % rispetto al proprio baseline denso**.

QUAL È DAVVERO IL BASELINE — E PERCHÉ QUESTO SEMPLIFICA IL PROGETTO

Il confronto *controllato* del paper **non è contro EAT-S**, ma contro il proprio **dense-transformer**: la stessa identica architettura che processa la sequenza dei frame temporali invece dei token latenti (96,67 %, 4,80 M, 0,645 G). Il numero di EAT-S è *citato dalla letteratura*, non rimisurato dagli autori.

Conseguenza operativa: il baseline da costruire per primo è **early convolution + dense transformer**, che condivide quasi tutto il codice col modello sparso e si ottiene cambiando solo il tokenizer. La reimplementazione completa di EAT-S, con tutte le sue augmentation, diventa **opzionale**.

DISTINZIONE DA AVERE PRONTA ALL'ORALE

Qui «sparse» significa **pochi token**, non **attenzione sparsa**. La complessità della self-attention è $O(n^2d)$ per la matrice di attenzione e $O(n d^2)$ per le proiezioni: questo lavoro attacca **n**, non il pattern di attenzione. I parenti concettuali sono TokenLearner, Adaptive Token Sampling e Perceiver — *non* gli Sparse Transformers di Child et al. (2019), Longformer o BigBird. Confondere le due cose è l'errore che un esaminatore cerca.

3 Il metodo, per intero

DA DOVE VIENE DAVVERO

L'ancestor del metodo è **SparseFormer** (Gao et al. 2023, arXiv:2304.03768, *Sparse visual recognition via limited latent tokens*), un modello per ImageNet: questo lavoro ne è la trasposizione all'audio. Gli altri due riferimenti strutturali sono **Faster R-CNN** (parametrizzazione delle regioni) e **AdaMixer** (pesi di decoding adattivi). **Le regioni non sono scelte per saliency né per energia: sono parametri appresi**, aggiustati iterativamente dai token stessi. Leggere SparseFormer non è opzionale — è il posto dove il metodo è spiegato per esteso.

Il modello ha due componenti: lo **sparse feature extractor** e il **classification transformer**.

3.1 Sparse feature extractor

Si parte da N token latenti $t \in \mathbb{R}^d$, ciascuno accoppiato a una regione $b = (x, y, w, h)$ nel piano tempo-frequenza. **Sia i token iniziali sia le regioni iniziali sono parametri del modello, appresi durante il training.** I centri sono inizializzati su una **griglia**, larghezza e altezza a metà della dimensione della feature map. Il blocco seguente si ripete L_{rep} volte:

Passo	Operazione	Formulazione
1	Aggiustamento della regione parametrizzazione Faster R-CNN	$(t_x, t_y, t_w, t_h) = \text{Linear}(t)$ $x' = x + t_x \cdot w \quad y' = y + t_y \cdot h$ $w' = w \cdot \exp(t_w) \quad h' = h \cdot \exp(t_h)$
2	Campionamento sparso P punti per token	$\{(\Delta x_i, \Delta y_i)\}_P = \text{Linear}(\text{LayerNorm}(t))$ $\tilde{x}_i = x + 0,5 \cdot \Delta x_i \cdot w \quad \tilde{y}_i = y + 0,5 \cdot \Delta y_i \cdot h$ offset normalizzati a tre deviazioni standard ; feature estratte per interpolazione bilineare $\rightarrow x^{(0)} \in \mathbb{R}^{P \times C}$
3	Decoding adattivo stile AdaMixer	$[M_c \mid M_s] = F(t), \quad M_c \in \mathbb{R}^{C \times C}, M_s \in \mathbb{R}^{P \times P}$ $F =$ due reti lineari con dimensione nascosta $d/4$ $x^{(1)} = \text{GELU}(x^{(0)} M_c)$ (mixing sui canali) $x^{(2)} = \text{GELU}(M_s x^{(1)})$ (mixing spaziale) $t' = t + \text{Linear}(x^{(2)})$ (update residuo del token)

IL DETTAGLIO CHE DECIDE SE IL MODELLO IMPARA O NO

Non si campiona direttamente dallo spettrogramma. Il gradiente che passa per l'interpolazione bilineare è troppo rumoroso quando i punti di campionamento sono pochi e locali. Si applica prima una **early convolution** (Xiao et al. 2021) e si campiona sul suo output. Saltare questo passaggio è il modo più diretto per ottenere un modello che non converge.

3.2 Classification transformer

La sequenza di N token latenti entra in uno stack di transformer-encoder. **Nessun positional encoding:** i token non hanno un ordine intrinseco. Si fa la media degli embedding finali e si applica un classificatore lineare.

La complessità della MHSA è $O(N^2)$ con $N \ll T$, dove T è il numero di frame temporali del modello denso: **è qui che sta tutto il guadagno.**

3.3 Configurazione e training

N token	P campioni	d _{token}	d _{encoder}	L _{rep}	L _{encoder}
4	36	64	128	3	8

Un layer lineare fa da ponte fra le due dimensioni (64 → 128). Training: **AdamW** ($\beta_1 = 0,9$, $\beta_2 = 0,99$, $\text{eps} = 10^{-8}$), lr **$3 \cdot 10^{-4}$** , schedule **one-cycle**, weight decay 10^{-5} , **EMA** 0,995, batch size **64**. Augmentation: *mixing* e *phasemix*, prese direttamente da EAT.

3.4 Le ambiguità: stato al 18 agosto 2026

QUESTA SEZIONE È ORA UN RIEPILOGO — IL RIFERIMENTO È IL MANUALE

Quasi tutte le ambiguità elencate qui sono state **risolte** leggendo il codice ufficiale di SparseFormer e di EAT. Il catalogo completo, con formule ed evidenze, sta in `Manuale_tecnico.pdf` §9. In caso di discordanza fra questo documento e il manuale, **fa fede il manuale.**

Ambiguità	Stato
Parametri dello spettrogramma — mel o lineare? <code>n_fft</code> ? hop? bin?	Risolta. Mel, giustificato da KWT e AST (<i>non</i> da EAT, che usa la forma d'onda grezza). Il numero di bin è quasi irrilevante: 0,8 punti di escursione su un fattore 4 di risoluzione. Adottati 64.
Contraddizione «96 dimensional» / «196 kernels»	Risolta dal conteggio dei parametri: <code>c196_proj96</code> , cioè 196 kernel seguiti da una proiezione 1×1 a 96 — la lettura in cui entrambe le cifre sono vere.
Stride del max pooling	Dedotto: (2, 1), solo sull'asse frequenza → 51 frame. Con (2, 2) il costo si ferma al 70 % di quello dichiarato.
Normalizzazione nello stem	Risolta dal codice di SparseFormer: LayerNorm sui canali <i>dopo</i> il pooling, nessuna BatchNorm.
Formule di <code>mixing</code> e <code>phasemix</code>	Risolte dal codice di EAT. Nessuna delle due era quello che il nome suggeriva.
Normalizzazione degli offset «a tre deviazioni standard»; inizializzazione	Risolte dal codice di SparseFormer. Da qui anche il perché N è sempre un quadrato perfetto.
Parametri di one-cycle, programmazione delle augmentation, weight decay	Risolti dal <code>trainer.py</code> di EAT.
Se i FLOPs di EAT-S (0,373 G) siano misurati o citati	Sono citati . Per confrontarsi con EAT-S vanno rimisurati col proprio contatore.
Restano aperte: il numero di epoche (nessuna fonte); lo scarto dell'11 % sui FLOPs (non spiegato, due candidati eliminati); <code>pool_freq</code> contro <code>flatten</code> (si addestrano entrambe); pre-norm contro post-norm (da abblare).	

UN'OSSERVAZIONE CRITICA GIÀ DISPONIBILE

Nella Tabella 3 del paper l'ablation sul numero di token **non è monotona**: 16 token danno 97,53 % ma 25 token danno 97,20 %, per poi risalire a 97,94 % con 36. I risultati sono su **singolo seed**. Questo suggerisce un rumore da seed dell'ordine di 0,3–0,4 punti, cioè paragonabile a diverse delle differenze discusse nel paper. È la giustificazione sperimentale dei tuoi ≥ 3 seed, ed è un'ottima osservazione critica da portare all'orale.

4 Pipeline realizzativa — sei fasi

Il principio che governa l'ordine: **l'infrastruttura di misura si costruisce prima del modello**, perché ciò che va riprodotto è un trade-off accuratezza/FLOPs, non un'accuratezza. Un contatore di FLOPs scritto dopo il modello tende ad essere costruito per confermare il risultato atteso.

Fase 0 — Sblocco

Obiettivo	Rimuovere tutte le dipendenze esterne dal progetto.
Azioni	PDF via proxy UNIFI fatto · leggere SparseFormer (arXiv:2304.03768), dove il metodo è spiegato per esteso · form risorse di calcolo · ricevimento del lunedì con una pagina di proposta (task, riduzioni previste, metriche) · repo privato Codeberg con <code>dl.unifi</code> invitato · ambiente Python verificato.
Criterio di uscita	Hai letto entrambi i paper più SparseFormer, hai parlato col docente, <code>torch.cuda.get_device_capability()</code> restituisce <code>(12, 0)</code> .

Fase 1 — Prerequisiti mirati

Obiettivo	Studiare i prerequisiti <i>nell'ordine in cui servono al codice</i> , non «tutto il libro dall'inizio».
Azioni	Segui il calendario di §6, che alterna un blocco teorico e il pezzo di codice che lo usa.
Criterio di uscita	Non è una fase separata: corre in parallelo a tutte le altre fino alla fine.

Fase 2 — Dati e misura fase critica

Obiettivo	Un dataset con split corretti e un contatore di FLOPs affidabile, <i>prima</i> di qualsiasi modello.
Azioni	Vedi il protocollo sperimentale dettagliato in §5.
Criterio di uscita	Sai caricare un batch in <50 ms, e sai misurare i FLOPs di un modulo arbitrario con due contatori indipendenti che concordano.

Fase 3 — Baseline: early convolution + dense transformer

Obiettivo	Riprodurre il dense-transformer degli autori: 96,67 %, 4,80 M parametri, 0,645 G FLOPs. È il baseline del confronto controllato, non EAT-S.
Azioni	Spettrogramma → early convolution (Xiao et al. 2021) → sequenza di frame temporali → stack di 8 transformer-encoder → average pooling → classificatore lineare. Ricetta: AdamW lr $3 \cdot 10^{-4}$, one-cycle, wd 10^{-5} , EMA 0,995, batch 64, mixing + phasemix.
Perché prima	Condivide tutto il codice a valle col modello sparso: cambia solo come si producono i token. Farlo per primo isola i bug del training loop da quelli del tokenizer.
Criterio di uscita	Un numero su test set riproducibile su 3 seed, con deviazione standard, e FLOPs misurati.

Fase 4 — Sparse feature extractor

Obiettivo	Il metodo di §3, reimplementato da zero: 96,88 %, 2,87 M, 0,055 G.
Struttura	Tre moduli intercambiabili via config, esattamente i tre passi di §3.1: (a) region adjustment (Linear + parametrizzazione esponenziale); (b) sparse sampling (offset appresi + interpolazione bilineare) — N e P sono le due manopole del trade-off; (c) feature decoding adattivo (M_C , M_S , update residuo). Il transformer è quello della Fase 3, riusato senza modifiche.
Ordine di implementazione	Prima il forward con regioni fisse su griglia (niente aggiustamento): verifica shape e FLOPs. Poi attiva l'aggiustamento iterativo. Così sai da dove viene un eventuale mancato apprendimento.
Deliverable chiave	La curva accuratezza vs FLOPs al variare di $N \in \{4, 9, 16, 25, 36\}$ e $P \in \{16, 36, 64, 128\}$ — cioè le due ablation della Tabella 3 del paper, ma con più seed . È il grafico centrale della presentazione.
Verifica qualitativa	Riproduci la Figura 2 : i punti campionati sovrapposti allo spettrogramma, un colore per token, ai tre stadi di L_{rep} . Dovresti vedere il campionamento passare da uniforme a concentrato. È la diapositiva che convince più di qualsiasi tabella.

Fase 4-bis — EAT-S

opzionale

Quando farlo	Solo dopo che Fasi 3 e 4 sono chiuse. Serve se vuoi confrontarti con EAT-S <i>con il tuo contatore di FLOPs</i> anziché citarne i numeri.
Azioni	Front-end conv 1D (depthwise + 1×1) → transformer → testa; ricetta EAT completa; ablation delle augmentation (nessuna → +mixup → +FreqMix/PhaseMix → completa), che è il contributo dichiarato del paper 1.
Sul codice ufficiale	Esiste. Consultalo per risolvere ambiguità, non per copiare, e dichiaralo nel README e all'orale.

Fase 5 — Esperimenti onesti

Obiettivo	Rendere i risultati difendibili sotto interrogatorio.
Azioni	≥3 seed per ogni configurazione riportata, con media e deviazione standard. Ablation sul criterio di selezione delle regioni: <i>la saliency appresa serve davvero, o basta l'energia?</i> — è la domanda scientificamente più interessante che puoi porre a questo paper.
Bonus ad alto rendimento	Il programma ha una voce dedicata all' <i>hyperparameter optimization</i> (lezioni 02/12 e 10/12). Usa Optuna con Hyperband/ASHA per tarare K e il learning rate: trasformi un pezzo di programma teorico in un pezzo del tuo progetto, e all'orale spieghi successive halving avendolo eseguito.
Regola	Meglio 3 configurazioni × 3 seed che 9 configurazioni × 1 seed.

Fase 6 — Presentazione e orale

Obiettivo	30 minuti che coprano letteratura, derivazione, esperimenti.
Struttura	Problema e letteratura (5') → metodo e derivazione (10') → esperimenti e risultati (10') → limiti e domande (5').
Criterio di uscita	Sai scrivere alla lavagna tutte le derivazioni di §8 senza appunti.

Criterio di successo — definirlo ora, non a posteriori

Livello	Condizione
Successo pieno	EAT-S entro ~1–2 punti dal valore riportato; sparse-transformer che, a parità di pipeline, perde ~1 punto rispetto a EAT-S tagliando i FLOPs di un fattore ≈ 5 –7; curva accuratezza/FLOPs coerente col claim.
Successo accettabile sufficiente per un ottimo voto	Numeri assoluti più bassi per budget ridotto, ma trend riprodotto e gap analizzato e spiegato con esperimenti.
Fallimento vero	Non saper dire <i>perché</i> i tuoi numeri differiscono da quelli pubblicati.

Un progetto che afferma «non ho riprodotto il 96,88 %, ho ottenuto il 94,1 %, ed ecco le tre cause candidate isolate con questi esperimenti» vale più di un numero identico ottenuto senza capire perché.

5 Protocollo sperimentale

5.1 Google Speech Commands V2 — protocollo confermato dal paper

105 829 clip, **35 classi**, 16 kHz, zero-padding a 1 secondo esatto. Il paper dichiara la ripartizione, e coincide con gli split ufficiali:

Split	Campioni	Uso
Training	84 843	addestramento
Validation	9 981	scelta degli iperparametri
Test	11 005	valutazione finale — una sola volta

Usa gli split ufficiali `validation_list.txt` e `testing_list.txt`, basati sull'hash dello speaker ID e quindi **speaker-disjoint**. Se i tuoi conteggi non danno esattamente 84 843 / 9 981 / 11 005, la pipeline dati è sbagliata: è un test di correttezza gratuito, sfruttalo. Split casuali gonfiano l'accuratezza di punti interi ed è un errore che verrà notato.

Il secondo dataset del paper è **ESC-50** (2 000 clip, 50 classi, 5-fold cross-validation, ricampionato da 44,1 a 22,05 kHz). È la parte *riducibile* del progetto: affrontala solo se Speech Commands è chiuso.

5.1-bis Target di riproduzione

Modello	Params	FLOPs	Accuracy	Nota
Sparse-transformer (N=4, P=36)	2,87 M	0,055 G	96,88 %	obiettivo della Fase 4
Dense-transformer	4,80 M	0,645 G	96,67 %	obiettivo della Fase 3
EAT-S	5,30 M	0,373 G	98,15 %	<i>citato</i> , non rimisurato dagli autori
HTS-AT	28,80 M	4,066 G	98,00 %	<i>citato</i>
SimPF (MobileNetV2)	3,40 M	0,244 G	95,60 %	<i>citato</i>

Verifica di coerenza già fatta: $0,055 / 0,373 = 0,147$, cioè **-85,3 %** rispetto a EAT-S; $0,055 / 0,645 = 0,085$, cioè **-91,5 %** rispetto al denso. I numeri del paper sono internamente consistenti.

5.2 Misura del costo computazionale

Il claim da riprodurre è **-85,25 % di FLOPs**: senza un contatore identico applicato ai due modelli non hai riprodotto niente.

- Misura su **un singolo input da 1 s, in inferenza**, e dichiara la convenzione usata (FLOPs vs MACs: differiscono di un fattore 2 — molti paper li confondono).
- **Due contatori indipendenti** che devono concordare: `torch.utils.flop_counter.FlopCounterMode` (nativo, nessuna dipendenza) e `fvcore`.
- Per il modello sparso i FLOPs dipendono da K: riporta il valore *teorico* e quello *misurato*, e segnala eventuali discrepanze.
- Riporta anche **parametri, latenza wall-clock e picco di memoria**: i FLOPs da soli non predicono il tempo reale, ed è un'osservazione che fa una buona diapositiva.

5.3 Prestazioni della pipeline dati

IL COLLO DI BOTTIGLIA NON È LA GPU

Pre-ricampiona tutto e salva in un unico tensore **memory-mapped** di forma `[N, 16000]` in `int16`, e sposta le **augmentation su GPU**. Ingombro su disco: **2,72 GB** per il training set ($84\,843 \times 16\,000 \times 2$ B), 0,32 GB per validation, 0,35 GB per test — circa 3,4 GB in totale. Non serve che stia in RAM: il sistema operativo tiene in page cache le parti calde.

Le augmentation su forma d'onda eseguite in CPU dentro il DataLoader sono ciò che ti farà credere di non avere abbastanza GPU — quasi sempre non è vero. Con 8 GB di VRAM il modello sparso (2,87 M parametri, input da 1 s) gira largamente entro i limiti al batch 64 dichiarato dal paper.

5.4 Riduzioni ammesse, in ordine di preferenza

1. Task a **12 classi** anziché 35;
2. sottocampionamento a ~30 % degli **speaker** (mantenendo lo split speaker-disjoint);
3. **8 kHz** anziché 16 kHz;
4. larghezza/profondità del transformer dimezzate.

In tutti i casi: **documenta la riduzione nel README** e mantieni intatto il protocollo di valutazione.

6 Piano di studio — dieci settimane

Ritmo consigliato per chi lavora da solo e senza scadenza fissata: **alternanza settimanale** fra un blocco teorico e il pezzo di codice che lo usa. È più lento sulla carta della sequenza «prima studio tutto, poi implemento», ma è l'unico modo per arrivare all'orale con le derivazioni davvero proprie.

Sett.	Studio	Codice
1	PDF Kavaki & Mandel · ImportantAug (arXiv:2112.07156) · Interspeech 2020	Ambiente · dataset SC-V2 · split ufficiali · cache memory-mapped
2	Autodiff (Baydin et al. 2017; Gebremedhin & Walther 2020) · conv arithmetic (Dumoulin & Visin 2016)	Front-end conv 1D · contatore FLOPs con doppia verifica
3	Attention e transformer (Vaswani et al. 2017; BB24 12.1) · einops (Rogozhnikov 2022)	Encoder transformer · EAT-S completo
4	Ottimizzatori (Kingma & Ba; Loshchilov & Hutter) · normalizzazione (Ioffe & Szegedy; Wu & He)	Training loop · one-cycle · EMA → primo numero su SC-V2
5	Data augmentation · mixup · BCE con target mixati · dropout (Srivastava et al. 2014)	Augmentation di EAT · ablation delle augmentation
6–7	Tokenizzazione e ViT (Dosovitskiy et al. 2020) · positional encoding · saliency	Sparse tokenizer · curva accuratezza/FLOPs al variare di K
8	HPO (Snoek et al. 2012; Li et al. 2018) · GP (RW06 1,2,4) · successive halving	Optuna + ASHA su K e learning rate
9–10	Ripasso trasversale degli argomenti non toccati dal progetto: GNN, U-Net e segmentazione, GP in dettaglio, contrastive/SimCLR, domain adaptation	Figure · README · presentazione · prove a voce

7 Il programma d'esame come checklist

Tabella di tutte le lezioni con le letture richieste (da studiare per l'esame) e la rilevanza per il progetto: alta = ci lavori direttamente · media = ti serve per capire · bassa = da studiare a parte.

Data	Argomenti	Letture	Progetto
17/09	Forme di apprendimento; cenni storici	BB24 1; GBC16 5.1	bassa
19/09	Supervised learning ed ERM; classificatore ottimo	BB24 4.1–4.2, 5.2–5.3; GBC16 5.5	media
24/09	Direzione generativa; LDA; MLE; ottimalità degli iperpiani; reti a singolo strato; MLE↔ERM	BB24 3.2, 5.1; GBC16 5.5, 5.7	media

Data	Argomenti	Lecture	Progetto
26/09	Loss per regressione (L2, L1, Huber); minimi quadrati; loss per classificazione (0-1, hinge, log); GLM; Bernoulli in exp family	BB24 4.1, 5.4	alta
01/10	GLM: regressione logistica; log-loss e likelihood; Gaussian e Poisson; softmax	BB24 5.4, 5.1; ZLLS23 4	alta
03/10	Geometria della softmax; log-sum-exp ; GD e SGD	BB24 7.1–7.2; ZLLS23 4, 12 · Bottou & Bousquet 2007	alta
08/10	GD vs SGD; tassi di convergenza; tradeoff del large scale learning; minibatch; momentum; Adagrad, RMSProp, Adam	BB24 7.2–7.3 · Bottou & Bousquet Hinton 2012 Kingma & Ba	alta
15/10	Feature learning ed end-to-end learning ; composizionalità; training layerwise; denoising autoencoder; MLP; rettificatori	BB24 6; GBC16 6, 6.1, 6.3–6.4, 14.2 · Bengio et al. 2013	alta
16/10	<i>Video 1</i> — framework, ambiente di sviluppo, tensori, lavoro remoto	ZLLS23 2.1–2.3	media
17/10	Espressività; basis function; reti RBF; Leaky/Parametric ReLU; MLP di ReLU come funzioni lineari a tratti; grafi di computazione	BB24 8, 9.1 · Baydin et al. 2017	media
22/10	Autodiff forward e reverse ; inizializzazione (LeCun, Glorot, He); regolarizzazione esplicita; ridge	BB24 8, 7.2.5, 9.1; GBC16 6.5, 7.1, 8.4; ZLLS 2.5, 5.4.2, 5.5 · Gebremedhin & Walther 2020	alta
23/10	<i>Video 2</i> — logistic regression in TensorFlow puro; Tensorboard; in PyTorch	ZLLS23 4	media
24/10	Interpretazioni del ridge; bias-variance e double descent ; interpretazione bayesiana e MAP; L1, lasso, elastic net	BB24 9.1–9.2, 9.3.2; GBC16 7.1–7.2, 7.8–7.9	media
25/10	Weight decay, AdamW ; weight sharing; early stopping; dropout ; GELU e relazione col dropout	BB24 7.2.5, 7.4, 9.1.3; GBC16 7.4, 8.4, 8.7.1 · Srivastava et al. 2014	alta
31/10	Data augmentation ; batch e layer normalization; CNN per segnali N-dimensionali e loro inductive bias	BB24 9.1.3, 7.4, 10.1; GBC16 9.1–9.2 · Ioffe & Szegedy 2015	alta
04/11	<i>Video 3</i> — autodiff in TensorFlow e PyTorch; MLP in TF/Keras	ZLLS23 2.5, 5	media
05/11	Equivarianza traslazionale; convoluzione, canali, stride, pooling ; convoluzioni dilatate e trasposte	BB24 10.1–10.2; GBC16 9.3–9.5; ZLLS23 7.2–7.6 · Dumoulin & Visin 2016	alta

Data	Argomenti	Lecture	Progetto
07/11	Bottleneck 1×1 ; norm per CNN (batch, layer, instance, group); gates; mixture of experts; skip connection, highway, residual; EfficientNet; segmentazione e U-Net; Dice loss	BB24 10.4–10.5; ZLLS23 9.4 · Wu & He 2018 Ronneberger et al. 2015	media
11/11	<i>Video 4</i> — dataloader in PyTorch ; CNN e DenseNet	ZLLS23 7 · Huang et al. 2017	alta
12/11	Problemi su sequenze; NLP di base; embedding; RNN e loro grafo di computazione; vanishing/exploding gradient	BB24 12.2; JM24 9; GBC16 10.1–10.2	media
14/11	RNN impilate e bidirezionali; LSTM e GRU ; seq2seq; encoder-decoder; decoding: greedy, Viterbi, beam search, sampling	BB24 11.3, 12.2; ZLLS23 10; JM24 8.4, 9.1 · Chung et al. 2014 Hochreiter & Schmidhuber 1997 Sutskever et al. 2014	bassa
19/11	Meccanismo di attenzione originale; soft dictionary; RNN con attention; traduzione automatica; attention per classificazione di sequenze	BB24 12.2 · Bahdanau et al. 2014	alta
21/11	Self-attention; parametrizzazione; complessità; multi-head; masking e batch matmul; transformer	BB24 12.1 · Vaswani et al. 2017 Rogozhnikov 2022 (einops)	alta
26/11	Positional encoding; Vision Transformer ; transfer learning; pretraining/fine-tuning; self-supervised e pretext task	BB24 12.1–12.4, 6.3 · Vaswani et al. 2017 Dosovitskiy et al. 2020	alta
02/12	<i>Video 5</i> — il problema dell'HPO; algoritmo elementare; introduzione ai processi gaussiani	B06 2.3, 3.3; RW06 1, 2, 4	media
05/12	Pretraining e fine-tuning in BERT; triplet loss; contrastive learning; SimCLR; domain adaptation; HPO model-based	JM24 11 · Devlin et al. 2018 van den Oord et al. 2019 Schroff et al. 2015 Chen et al. 2020	bassa
10/12	HPO model-based ed expected improvement; approcci multi-fidelity; successive halving, Hyperband , ASHA; meta-learning; domain adaptation; reweighting; mixup	Snoek et al. 2012 Bergstra et al. 2013 Li et al. 2018	alta

COPERTURA

Il progetto tocca direttamente o indirettamente la grande maggioranza del programma. Restano scoperti **cinque argomenti**, da studiare a parte nelle settimane 9–10: **graph neural network, U-Net e segmentazione semantica, processi gaussiani in dettaglio, contrastive learning e SimCLR, domain adaptation e generalization**. Sono cinque isole, non un programma intero.

8 Le derivazioni da saper fare alla lavagna

La pagina del corso è esplicita: «if the method uses an optimizer, which happens with overwhelming probability, **you are supposed to know how it works**». Tieni un quaderno di derivazioni a mano parallelo al codice: ogni volta che scrivi una riga del training loop, deriva la cosa corrispondente.

Derivazione	Perché proprio questa
Gradiente della cross-entropy con softmax: $\partial L / \partial z = p - y$	È la testa del tuo classificatore; il risultato pulito è il motivo per cui si usano insieme.
Il trick del log-sum-exp	Lezione dedicata (03/10) e problema numerico reale nel tuo codice.
Update di Adam con bias correction; differenza fra L2 e weight decay disaccoppiato (AdamW)	EAT usa AdamW. Domanda quasi certa: «perché in Adam L2 e weight decay non coincidono?»
Complessità della self-attention: $O(n^2d)$ per l'attenzione, $O(n d^2)$ per le proiezioni	È la giustificazione dell'intero paper 2. Devi saper dire per quali n domina quale termine, e quindi quanto guadagni davvero riducendo n .
Perché lo scaling $1/\sqrt{d_k}$	Domanda standard su Vaswani et al.
Backprop attraverso una convoluzione 1D, e il caso depthwise	È il front-end di EAT.
Reverse-mode AD: costo $O(1) \times$ il forward; perché reverse per $f: \mathbb{R}^n \rightarrow \mathbb{R}$ e forward nel caso opposto	Due lezioni e due letture richieste (Baydin; Gebremedhin & Walther).
Dimensione dell'output e receptive field di una convoluzione con stride e dilation	Lettura richiesta (Dumoulin & Visin); ti serve per dimensionare il front-end.
BatchNorm : forward, backward, differenza train/eval, e perché aiuta	Lettura richiesta (Ioffe & Szegedy); la spiegazione «internal covariate shift» è contestata — sappilo.
Mixup : perché BCE e non CE con target mescolati; lettura come regolarizzatore	Scelta esplicita nella ricetta di EAT: devi saperla motivare, non solo copiare.
Dropout come ensemble implicito / regolarizzazione adattiva	Lettura richiesta (Srivastava et al. 2014).
Bias-variance e double descent	Lezione 24/10; domanda tipica sui regimi sovraparametrizzati.
Posterior di un GP ed expected improvement	Se usi Optuna devi saper spiegare cosa c'è sotto.
Successive halving e Hyperband: allocazione del budget	Lettura richiesta (Li et al. 2018); lo esegui davvero in Fase 5.
Gradiente attraverso l'interpolazione bilineare rispetto alle <i>coordinate</i> di campionamento	È il meccanismo che rende differenziabile la selezione delle regioni — cioè il cuore del paper. Devi anche saper spiegare <i>perché</i> quel gradiente è rumoroso con pochi punti, e quindi perché serve la early convolution.

Derivazione	Perché proprio questa
Perché $w' = w \cdot \exp(t_w)$ e non $w' = w + t_w$	La forma esponenziale garantisce larghezze positive e rende l'aggiornamento moltiplicativo, quindi invariante di scala. Viene da Faster R-CNN: domanda naturale in sede d'esame.
Perché il decoding adattivo (M_C , M_S generati da t) e non un semplice layer lineare	Il paper dice esplicitamente che «simply using a linear layer is not effective». Saper spiegare la differenza fra pesi <i>fissi</i> e pesi <i>condizionati sul token</i> è il punto concettuale di AdaMixer.

9 Presentazione: struttura e domande probabili

Minuti	Sezione	Contenuto
0–5	Problema e letteratura	Classificazione audio; perché end-to-end vs spettrogramma; perché il costo dei transformer su audio è un problema; dove si collocano i due paper.
5–15	Metodo e derivazione	EAT: front-end e augmentation. Sparse-Transformer: scoring, sampling, decoding. La derivazione della complessità come filo conduttore.
15–25	Esperimenti	Protocollo (split ufficiali, seed, contatori di FLOPs) → tabella dei risultati tuoi accanto a quelli pubblicati → curva accuratezza/FLOPs → ablation.
25–30	Limiti e domande	Cosa non hai riprodotto e perché; ambiguità del paper e come le hai risolte.

DOMANDE DA ASPETTARSI

- «Sparse in che senso? Attenzione sparsa o pochi token?» → §2.2.
- «Come funziona l'ottimizzatore che hai usato?» → derivazione di AdamW alla lavagna.
- «Perché lo split ufficiale e non uno casuale?» → speaker-disjoint, leakage fra train e test.
- «FLOPs o MACs? Misurati come?» → convenzione dichiarata, due contatori concordi.
- «Il guadagno in FLOPs si traduce in latenza reale?» → i tuoi numeri di wall-clock; spesso no, ed è interessante spiegare perché.
- «Perché il tuo numero differisce da quello del paper?» → le cause candidate isolate sperimentalmente.
- «L'augmentation X serve davvero?» → la tua ablation.

10 Struttura del repository e consegna

```

audio-sparse-transformer/
  README.md           # comandi, seed, versioni, tempi, tabella risultati
                      # tuoi ACCANTO a quelli dei paper – unico documento valutato
  requirements.txt     # vincoli di installazione
  requirements.lock.txt # pip freeze dopo la prima installazione riuscita
  configs/            # un YAML per ogni esperimento riportato
  src/
    data/             # download, split ufficiali, cache mmap, augmentation
    models/           # eat.py, sparse_tokenizer.py, transformer.py
    train.py eval.py flops.py
  scripts/            # run_baseline.sh, run_sparse.sh, run_ablation.sh
  tests/              # test del tokenizer e delle shape
  results/            # CSV/JSON dei run + notebook che genera le figure

```

- Repo **privato** su Codeberg, con `dl.unifi` invitato come membro.
- **Nessun dato nel repo:** solo il link al dataset.
- Nel README dichiara esplicitamente **cosa hai consultato del codice ufficiale di EAT** e cosa hai scritto da zero.

11 Ambiente di sviluppo

GPU	NVIDIA GeForce RTX 5050 Laptop, 8151 MiB, driver 592.82 — architettura Blackwell, compute capability sm_120
Python	3.12.4 di sistema, venv dedicato al progetto
Stack verificato	torch 2.9.1+cu128 (CUDA 12.8, arch list include sm_120) · torchaudio 2.9.1

DUE TRAPPOLE GIÀ IDENTIFICATE

1. Wheel sbagliata. Le build PyTorch di default (CUDA ≤ 12.6) arrivano a `sm_90` e falliscono a runtime con `no kernel image is available for execution on the device` — un errore che sembra un driver rotto e non lo è. Installa dall'indice `cu128` e verifica che `torch.cuda.get_device_capability()` restituisca `(12, 0)` prima di scrivere altro.

2. Backend audio mancante. torchaudio 2.9+ ha dismesso l'I/O interno: senza `soundfile` installato non carichi i `.wav` di Speech Commands. Il suffisso `+cpu` di torchaudio invece *non* è un problema: le trasformate sono composte da operazioni PyTorch normali e girano su tensori CUDA.

Perché un venv dedicato e non un ambiente esistente: il progetto è valutato sulla riproducibilità e l'unico documento consegnato è codice più istruzioni. Un `pip freeze` prodotto da un ambiente generalista con JAX, OpenCV, mediapipe e pyautogui genera un `requirements.txt` privo di significato, che il docente non può usare per riprodurre nulla. Un ambiente dedicato con versioni pinnate è parte del deliverable.

12 Riferimenti

PAPER DEL PROGETTO

- Gazneli, A., Zimerman, G., Ridnik, T., Sharir, G., & Noy, A. (2022). *End-to-End Audio Strikes Back: Boosting Augmentations Towards An Efficient Audio Classification Network*. arXiv:2204.11479. Codice: github.com/Alibaba-MIIL/AudioClassification
- Kavaki, H. S., & Mandel, M. I. (2025). *Audio Sparse-Transformer for Speech Classification*. ICASSP 2025, 1–5. DOI 10.1109/ICASSP49660.2025.10890475 — **closed access**

ANCESTORS INDISPENSABILI — DA CUI VIENE IL METODO (§3)

- Gao, Z., Tong, Z., Wang, L., & Shou, M. Z. (2023). *SparseFormer: Sparse Visual Recognition via Limited Latent Tokens*. arXiv:2304.03768. — Il modello di cui il paper 2 è la trasposizione all'audio. **Lettura non opzionale.**
- Gao, Z., Wang, L., Han, B., & Guo, S. (2022). *AdaMixer: A Fast-Converging Query-Based Object Detector*. CVPR 2022. — Origine del decoding adattivo (M_c , M_s).
- Ren, S., He, K., Girshick, R., & Sun, J. (2015). *Faster R-CNN*. NeurIPS 28. — Origine della parametrizzazione delle regioni.

- Xiao, T., Singh, M., Mintun, E., Darrell, T., Dollár, P., & Girshick, R. (2021). *Early Convolutions Help Transformers See Better*. NeurIPS 34. — Il front-end senza cui il campionamento non impara.

CONTESTO: I MODELLI CON CUI IL PAPER SI CONFRONTA

- Warden, P. (2018). *Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition*. arXiv:1804.03209. — Il dataset e i suoi split ufficiali.
- Berg, A., O'Connor, M., & Tairum Cruz, M. (2021). *Keyword Transformer*. arXiv:2104.00769 — 97,7 % su SC V2.
- Gong, Y., Chung, Y.-A., & Glass, J. (2021). *AST: Audio Spectrogram Transformer*. arXiv:2104.01778 — 98,1 % con pretraining ImageNet.
- Chen, K., et al. (2022). *HTS-AT: A Hierarchical Token-Semantic Audio Transformer*. ICASSP 2022 — 98,0 %.
- Smith, L. N. (2018). *A Disciplined Approach to Neural Network Hyper-Parameters, Part 1*. arXiv:1803.09820. — Lo schedule one-cycle usato da entrambi i paper.
- Tarvainen, A., & Valpola, H. (2017). *Mean Teachers Are Better Role Models*. NeurIPS 30. — L'EMA dei pesi.
- Tokozume, Y., Ushiku, Y., & Harada, T. (2017). *Learning from Between-Class Examples for Deep Sound Recognition*. arXiv:1711.10282. — Il mixing in ambito audio.

TESTI DEL CORSO

BB24	Bishop & Bishop (2024), <i>Deep Learning: Foundations and Concepts</i> , Springer — bishopbook.com
GBC16	Goodfellow, Bengio & Courville (2016), <i>Deep Learning</i> , MIT Press — deeplearningbook.org
P24	Prince (2024), <i>Understanding Deep Learning</i> , MIT Press — udlbook.com
ZLLS23	Zhang, Lipton, Li & Smola (2023), <i>Dive into Deep Learning</i> — d2l.ai
JM24	Jurafsky & Martin (2025), <i>Speech and Language Processing</i> , 3 ^a ed. draft
B06	Bishop (2006), <i>Pattern Recognition and Machine Learning</i>
RW06	Rasmussen & Williams (2006), <i>Gaussian Processes for Machine Learning</i> , MIT Press

Il testo completo dei paper linkati dalla pagina del corso è accessibile da IP UNIFI; dall'esterno usare il proxy proxy-auth.unifi.it:8888 con le proprie credenziali.

Documento di lavoro personale per l'esame B031278 — Deep Learning, Fall 2025, Università di Firenze. I dati bibliografici e i risultati numerici sono tratti dalla pagina ufficiale del corso, dal preprint arXiv di Gazneli et al. e dal testo integrale di Kavaki & Mandel (ICASSP 2025). Le formule di §3 sono trascritte dal paper; le ambiguità di §3.4 sono punti su cui il paper non si pronuncia e che vanno risolti e dichiarati in fase di implementazione.